MiAbarrote: Yape, Vercel y MacroDroid

Guía de puesta en marcha | 14 de octubre de 2026

Sitio indicado: https://mi-abarrote.vercel.app/

Objetivo: recibir en MiAbarrote el titular y el importe de una notificación de cobro de Yape Bolivia. El teléfono envía un POST; Vercel lo valida y almacena en Redis; la caja muestra el aviso y el cajero confirma la venta.

Orden recomendado: configurar Redis y Vercel, conectar la caja, probar un envío manual desde MacroDroid y, finalmente, activar la captura de Yape. No hace falta mantener encendida una computadora que actúe como servidor.

Esta guía no contiene contraseñas. Los textos `TU_CLAVE_WEBHOOK` y los valores de ejemplo deben sustituirse en tu configuración privada.

El ejemplo facilitado fue «Eduardo David Lucas Choque 230Bs o algo así». Yape confirma oficialmente las notificaciones de cobro, pero no publica en esa página un contrato del texto exacto. El extractor de esta guía es un ejemplo estricto para ese formato: debes comprobar el texto real antes de habilitar el envío automático. Fuente: https://www.yape.com.bo/yapenegocios/

1. Preparar Redis y las claves

1. Entra en https://console.upstash.com/ con tu cuenta y crea o selecciona una base Redis. Si tienes una base existente para este proyecto, úsala. Comprueba sus límites y consumo en el panel.

2. En los datos de conexión REST copia la URL HTTPS y el token con permisos de lectura y escritura. Son `UPSTASH_REDIS_REST_URL` y `UPSTASH_REDIS_REST_TOKEN`. No copies la dirección TCP ni un token de solo lectura.

3. Genera tres claves distintas, aleatorias y de al menos 32 caracteres. Puedes ejecutar esta orden tres veces en una terminal con Node.js y guardar cada resultado en un gestor de contraseñas:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

4. Asigna una clave a cada propósito. `PAYMENT_WEBHOOK_KEY` irá al teléfono; `PAYMENT_READER_KEY` la introduce el cajero; `PAYMENT_SESSION_SECRET` se queda exclusivamente en Vercel. No uses el PIN de Yape como ninguna de estas claves.

5. Reserva el nombre `miabarrote-produccion` para `PAYMENT_NAMESPACE`. Si usas despliegues Preview, dales otro nombre y otras credenciales para que las pruebas no compartan cobros con producción.

Redis conserva las notificaciones y sus asociaciones durante 24 horas. La reserva de una notificación dura cinco minutos y se renueva mientras el modal permanece abierto. El teléfono y la caja deben tener internet.

Resultado esperado: tienes dos datos REST y tres claves privadas diferentes, sin subirlos a GitHub.

Fuente técnica: https://upstash.com/docs/redis/features/restapi

2. Configurar el proyecto en Vercel

1. Abre https://vercel.com/dashboard y selecciona el proyecto asociado a `unifor62-max/MiAbarrote`. En Settings > Git comprueba que Production Branch sea `main`.

2. En Settings > Build and Deployment comprueba Framework Preset `Vite`, Root Directory la raíz del repositorio (vacío o `.`), Node.js `22.x`, Install Command `npm ci`, Build Command `npm run build` y Output Directory `dist`. El repositorio ya contiene esos comandos en `vercel.json`.

3. En Settings > Environment Variables añade las siguientes variables al entorno Production:

Variable	Valor
VITE_PAYMENT_MODE	cloud
UPSTASH_REDIS_REST_URL	URL REST HTTPS de tu base
UPSTASH_REDIS_REST_TOKEN	Token REST de lectura y escritura
PAYMENT_WEBHOOK_KEY	Primera clave aleatoria
PAYMENT_READER_KEY	Segunda clave aleatoria
PAYMENT_SESSION_SECRET	Tercera clave aleatoria
PAYMENT_NAMESPACE	miabarrote-produccion

4. Guarda las variables. En Deployments selecciona el último despliegue de `main` y pulsa Redeploy. Un cambio de variables no modifica un despliegue ya construido. `VITE_PAYMENT_MODE` debe estar configurada antes del build.

5. Espera a que el estado sea Ready. En Settings > Domains confirma que `mi-abarrote.vercel.app` corresponde a este proyecto y a Production.

6. Usa siempre el dominio de producción en MacroDroid. Si el envío devuelve una página HTML de login de Vercel, revisa Deployment Protection: MacroDroid necesita poder acceder a la ruta mediante HTTPS. Si tu proyecto exige protección adicional, configura el acceso automatizado permitido por Vercel antes de continuar; la clave del webhook seguirá siendo obligatoria.

No publiques las claves como variables `VITE_*`: ese prefijo expone su valor en el navegador. La única variable pública de esta tabla es `VITE_PAYMENT_MODE`.

Sin estas variables, la interfaz puede desplegarse en modo demo o las funciones devolverán error de configuración. Un deploy Ready por sí solo no prueba la recepción de pagos.

Fuentes: https://vercel.com/docs/environment-variables/managing-environment-variables y https://vercel.com/docs/functions/runtimes/node-js

3. Conectar la caja y comprobar las rutas

1. Abre https://mi-abarrote.vercel.app/ en el navegador de la caja e inicia la sesión demostrativa de la aplicación.

2. En Ventas pulsa Conectar pagos. Introduce el valor de `PAYMENT_READER_KEY`, no la clave del teléfono. Espera el estado de conexión.

3. La conexión dura ocho horas y se guarda mediante una cookie segura. Si expira o cambias de navegador, vuelve a conectar. Usa una sola caja en esta fase.

4. Abre Registrar venta, selecciona productos y entra en el paso 2. Selecciona QR. Anota el total exacto en bolivianos: lo usarás en la prueba del teléfono.

Las funciones publicadas son `/api/webhook-pago`, `/api/pagos-sesion` y `/api/pagos-estado`. No hay que añadir un servidor Express, configurar CORS abierto ni instalar Redis en el teléfono.

Un GET en `/api/webhook-pago` no es una prueba válida de cobro: esta ruta requiere POST JSON con autorización. Las rutas de lectura requieren la sesión de caja. Una respuesta de error JSON de autenticación indica que alcanzaste la función; una página HTML del sitio o un 404 indica que debes revisar el despliegue y la raíz del proyecto.

El primer sondeo inicia el cursor: no recupera pagos anteriores a la conexión. Un pago recibido con el modal cerrado o antes del paso 2 muestra el aviso, pero no se aplica después a otra venta. Por eso conecta primero, abre el paso 2 y luego envía la prueba.

Si aparece Simular envío MacroDroid, revisa `VITE_PAYMENT_MODE=cloud` y reconstruye en Vercel. La simulación no está disponible en recepción real.

Resultado esperado: caja conectada y paso 2 esperando un QR del importe exacto.

4. Probar HTTP Request desde el teléfono

1. Instala o actualiza MacroDroid desde su distribución oficial. Usa una versión que incluya JavaScript con `mdOutput` y diccionarios JSON; esas funciones se incorporaron en la versión 5.54. Los nombres traducidos de los menús pueden variar.

2. Crea una macro llamada Prueba MiAbarrote. Usa un disparador manual, como un acceso directo o un botón de MacroDroid. Todavía no selecciones Yape.

3. Añade una acción HTTP Request / Solicitud HTTP y configura:

Campo	Configuración
Método	POST
URL	https://mi-abarrote.vercel.app/api/webhook-pago
Header	Content-Type: application/json
Header	Authorization: Bearer TU_CLAVE_WEBHOOK
Cuerpo	Texto JSON, mostrado abajo
Respuesta	Guardar código HTTP y cuerpo en variables locales

4. En el header Authorization sustituye `TU_CLAVE_WEBHOOK` por `PAYMENT_WEBHOOK_KEY`. Conserva un espacio después de `Bearer`. No uses la clave de caja.

5. Pega este cuerpo. Sustituye 230 por el total que anotaste. Cambia el ID en cada prueba nueva; conserva el mismo ID y cuerpo si estás reintentando la misma prueba:

```
{
  "id": "prueba-telefono-001",
  "titular": "Cliente de prueba",
  "monto": 230,
  "timestamp": "2026-10-14T10:00:00-04:00"
}
```

6. Guarda y ejecuta la macro mientras la caja está en el paso 2 QR. Debes recibir HTTP `201`, un aviso con el titular y un desglose QR igual al total. Pulsa Confirmar y abre el comprobante.

7. Reenvía exactamente el mismo cuerpo: debe responder `200` con `duplicate: true` y no crear otro pago. Para otra venta usa otro ID y envía después de entrar nuevamente al paso 2.

El JSON exige monto numérico, positivo y con hasta dos decimales. No envíes `"230Bs"`, `"230"` ni separadores de miles. El timestamp debe ser ISO con zona; en la macro automática será dinámico. La fecha fija de arriba sirve solo para la prueba del transporte.

Fuente: https://macrodroidforum.com/wiki/index.php/Action%3A_HTTP_Request

5. Capturar la notificación real de Yape

1. En Android activa las notificaciones de Yape. En MacroDroid autoriza Acceso a notificaciones cuando lo solicite. Permite a MacroDroid funcionar en segundo plano y revisa las restricciones de batería de tu fabricante si las macros se detienen con la pantalla apagada.

2. Crea una macro de diagnóstico Leer cobro Yape. Disparador: Notificación recibida, aplicación Yape, inicialmente sin filtro de texto. No selecciones todas las aplicaciones ni notificación eliminada.

3. Añade una acción para mostrar un diálogo local con el título y el texto de la notificación. Insértalos desde el selector de texto mágico del disparador: Título de notificación (`{not_title}`) y Texto de notificación (`{notification}`). No envíes todavía un HTTP Request.

4. Comprueba con un cobro real autorizado. Anota el título y el cuerpo exactos, incluidos orden, moneda, puntos y comas. Si Yape oculta el nombre o importe, revisa sus opciones de notificaciones y el contenido que Android entrega a MacroDroid.

5. Repite con una notificación de operación enviada y una publicidad, si ya tienes ejemplos. El filtro final debe aceptar únicamente cobros entrantes, usando el título/texto real observado. Un filtro que solo busque `Bs` también puede aceptar gastos o promociones.

6. Desactiva el diálogo de diagnóstico después de configurar el filtro. No compartas ni publiques los nombres reales del registro de prueba.

Yape Bolivia confirma que emite avisos inmediatos de cobro. Su página pública no especifica la plantilla exacta del mensaje ni garantiza los mismos textos entre versiones. No se verificó tu teléfono desde esta sesión; por eso este paso es obligatorio antes de automatizar.

Si aparecen avisos de resumen de varias operaciones, exclúyelos. Si el disparador permite ignorar notificaciones repetidas, usa esa opción solo después de verificar que no oculta dos cobros distintos del mismo importe. Para deduplicación bancaria sólida necesitas una referencia estable de transacción cuando Yape la proporcione; un GUID nuevo no elimina dos avisos del mismo cobro.

Fuentes: https://www.yape.com.bo/yapenegocios/ y https://www.macrodroidforum.com/wiki/index.php?title=Trigger%3A_Notification

6. Construir el envío automático

Realiza estos pasos en la macro de Yape, manteniéndola desactivada hasta terminar las pruebas. El extractor siguiente acepta únicamente el ejemplo Nombre completo 230Bs (también 230,50Bs o 230.50 Bs). Si el aviso real dice algo distinto, adapta el patrón con el texto del paso 5; no borres la validación para que acepte cualquier mensaje.

1. Crea una variable local de tipo Diccionario llamada `entrada`. Con acciones Establecer variable, añade `texto` de tipo cadena, con el texto mágico `{notification}`, e `id` de tipo cadena. Para `id` inserta Crear GUID desde el menú de texto mágico, una sola vez por ejecución.

2. Crea una variable local de tipo cadena llamada `cuerpoPago`. Añade una acción JavaScript y selecciona esa variable como resultado. Pega este código. Inserta `{lvjson=entrada}` mediante el selector de variables en formato JSON si tu editor no lo reconoce al escribirlo:

```
var entrada = {lvjson=entrada};
var texto = String(entrada.texto || "").trim();
var m = /^(.+?)\s+(\d+(?:[.,]\d{1,2})?)\s*Bs\.?$/i.exec(texto);
var cuerpo = "";
if (m) {
  var nombre = m[1].trim();
  var monto = Number(m[2].replace(",", "."));
  if (nombre && nombre.length <= 120 && monto > 0) {
    cuerpo = JSON.stringify({
      id: entrada.id,
      titular: nombre,
      monto: monto,
      timestamp: new Date().toISOString()
    });
  }
}
mdOutput(cuerpo);
```

3. Añade un bloque Si: `cuerpoPago` no está vacío. Dentro coloca la acción HTTP Request del paso 4; en el cuerpo sustituye el JSON fijo por `{lv=cuerpoPago}`. Fuera de ese bloque no debe enviarse nada. Si está vacío, muestra un aviso local «Formato Yape no reconocido» y revisa el patrón.

4. Mantén los mismos headers, URL y variables de respuesta. `JSON.stringify` escapa nombres correctamente; no construyas JSON concatenando el texto de la notificación ni incrustes `{notification}` directamente dentro del código JavaScript.

5. Para probar el extractor antes de activarlo, pon temporalmente en `entrada.texto` el ejemplo `Eduardo David Lucas Choque 230Bs`. Debe producir titular `Eduardo David Lucas Choque`, monto numérico `230` y una fecha ISO terminada en Z. Luego restaura el texto mágico del disparador.

6. Guarda el filtro de cobros entrantes del paso 5, activa la macro y verifica un cobro real. Comprueba en la caja el importe y el titular antes de confirmar.

Para reintentar, repite solamente HTTP Request con el mismo `cuerpoPago`; no vuelvas a ejecutar JavaScript ni Crear GUID. Reintenta fallos de red/503 de forma limitada; respeta Retry-After en 429. En 400/401/409 corrige la causa antes de reenviar. No reutilices una única variable global de cuerpo para múltiples cobros simultáneos: podrían sobrescribirse.

Referencia de estas funciones de MacroDroid:
https://www.macrodroidforum.com/index.php?threads%2Fv5-54.10140%2F=

7. Verificar cobros y resolver problemas

1. Efectivo: QR 0 y efectivo igual al total; Confirmar debe funcionar sin notificación.

2. QR exacto: abre paso 2 antes del aviso; un importe igual al total habilita Confirmar. Un importe distinto muestra discrepancia y no vende.

3. Híbrido: el cobro QR debe ser menor al total. La diferencia se asigna a efectivo. Si editas los importes como contingencia, ambos deben ser positivos y sumar el total; el comprobante identifica el registro manual.

4. Segundo pago: muestra otro aviso sin sustituir el primero asociado. Usa Cambiar pago para liberar la asociación y esperar uno nuevo. Cancelar descarta el borrador.

5. Comprobante: revisa método, QR, efectivo y titular asociado. La confirmación es siempre manual.

Síntoma	Revisión
400	JSON inválido, monto como texto, titular vacío o timestamp sin zona
401	Bearer/clave webhook incorrectos; o sesión de caja vencida
409	Mismo ID con datos diferentes o reserva no disponible
429	Límite de intentos; esperar el Retry-After
503	Variables privadas/configuración o Redis no disponible; revisar logs de Vercel
HTML/login/404	Dominio, raíz del proyecto, rutas API o Deployment Protection
Aviso, pero no se aplica	Llegó antes de paso 2, modo Efectivo, discrepancia o ya hay pago asociado
No llega al apagar pantalla	Internet, acceso a notificaciones y batería/segundo plano en Android

En Vercel abre el despliegue y sus logs de funciones. No pegues claves, cuerpos bancarios completos ni datos de clientes en tickets públicos. Si una clave queda expuesta, reemplázala en Vercel y en el dispositivo correspondiente, y redepliega.

Lista final: deploy Ready; variables Production guardadas; caja conectada; HTTP 201 desde teléfono; reintento 200 duplicado; filtro solo de cobros; titular/importe real correctos; comprobante correcto.

Límite de esta fase: las ventas e inventario siguen en memoria y se pierden al recargar. Redis persiste únicamente notificaciones/asociaciones; recargar tras consumir un pago puede dejarlo consumido sin comprobante recuperable. La integración acredita el mensaje enviado por MacroDroid, no una conciliación directa con el banco. Usa una sola caja y no recargues durante la confirmación.